```
package main

/* Implements datatypes:
pstackItem           An item on the parse stack for an SR parser
parseStack           A parse stack for an SR parser.
*/

/* Stack operations implemented:
1. function newParseStack()         Returns new empty parse stack.
2. method push(itm pstackItem)      Pushes item on stack.
3. method top()                     Returns the top of the stack. No side-
effect.
4. method pop()                     Pops the stack and returns the popped item.
5. method popnum(n int)             Pops n items from stack. No return value.
6. method String()                  Returns a string representation of the
stack.
*/

import (
      "container/list"
)

/* pstackItem
==============*/
type pstackItem struct {
      grammarSym, stateSym string
}

/* String()
Implement a String() method with this exact signature.
The print statements in the "fmt" package will understand
this and use it to print instances of this datatype.
*/
func (se pstackItem) String() string {
      return se.grammarSym + se.stateSym
}

/* parseStack: list implementation
==================================*/
type parseStack struct {
      stack *list.List
}

/* newParseStack()
Creates and returns a new empty stack
*/
func newParseStack() parseStack {
      ps := parseStack{}
      ps.stack = list.New()
      return ps
}

func (stk parseStack) push(itm pstackItem) {
      stk.stack.PushFront(itm)
}

/* top()
Returns top of the stack. No side-effect.
*/
func (stk parseStack) top() pstackItem {
      e := stk.stack.Front()
      return e.Value.(pstackItem)
}
```

```
func (stk parseStack) pop() pstackItem {
	e := stk.stack.Front()
	if e != nil {
		stk.stack.Remove(e)
		return e.Value.(pstackItem)
	}
	return pstackItem{"", ""}
}

func (stk parseStack) popNum(n int) {
	for i := 1; i <= n; i++ {
		stk.pop()
	}
}

func (stk parseStack) len() int {
	return stk.stack.Len()
}

func (stk parseStack) String() string {
	str := ""
	if stk.stack.Len() > 0 {
		e := stk.stack.Back()
		str = e.Value.(pstackItem).String()
		for e.Prev() != nil {
			str += e.Prev().Value.(pstackItem).String()
			e = e.Prev()
		}
	}
	return str
}
```